server to create a new container using the `hello-world` image, the output came from the container executing according to its configuration (and sending the output back to the parent shell). Although it is tempting to think of containers as being virtual machines they are not: containers should be conceptualized as single processes running in isolated environments, not general purpose machines.

Docker images are stored locally, if your `docker run` command specifies an image that can't be found locally Docker will attempt to download it from a repository. The default repository is `https://hub.docker.com` and the vast majority of official images for all kinds of software (including PHP) can be found there. When first running `hello-world` you might have noticed some informational output from Docker about locating and downloading the image:

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
719385e32844: Pull complete
Digest: sha256:4f53e2564790c8e7856ec08e384732aa38dc43c52f02952483e
3f003afbf23db
Status: Downloaded newer image for hello-world:latest
```

Images define a root filesystem for containers which consist of a series of read-only layers composed to form a single virtual filesystem (Docker uses OverlayFS for this). Multiple images can share the same set of layers, and multiple containers can read from the same layers when executing: this optimizes host disk usage compared to virtual machines. A new layer is only created when a change to the container filesystem occurs, and the new layer contains only the changes so it is usually very lightweight.

After successfully running the `hello-world` image, we can see that a copy of it stays on our machine:

```
$ docker image ls --filter="reference=hello*"
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
hello-world         latest              9c7a54a9a43c        4 months ago        13.3kB
```

The `--filter="reference=hello*"` filters the results based on repository name, just in case you're an old Docker hand and happen to have lots of other images already present on your system (in which case you probably skip the rest of this section). We can learn about the image by running `docker image inspect hello-world`, which dumps a lot of image metadata in a JSON format: